AI for Forensics and Capstone Intelligence

Student Name: Fuseini Imoru Kantuogaa

Student ID: C11/26/DFIT/17291

Instructor: Amin Iddris

Case ID: ICDFA-DFIR-AI-026
Lab Reference: CIP-B107-LAB01
Date of Collection: 2026-10-27 14:34:29 WAT (13:34:29 UTC)
Environment: Python 3.x, scikit-learn, pandas, matplotlib, Kali Linux (venv)
Random Seed: 42

1. Remit

This memorandum documents the design, execution, and validation of an AI-assisted triage workflow for identifying candidate IPv4 evidence from a simulated mobile-device configuration export. The examination distinguishes candidate IP evidence from misleading text (software versions, malformed numbers, address-like fragments) and presents defensible findings to an intelligence review panel.

Critical caveat: The supplied files are teaching artifacts. No recovered address is classified as malicious without corroborating evidence. "Positive" denotes the source annotation of an intended IP evidence candidate, not proof of routability, safety, public status, or malicious activity.

2. Evidence Manifest and Chain of Custody

All original evidence was preserved in an isolated originals directory with restricted permissions (files: 444; directory: 555). SHA-256 hashes were computed immediately upon extraction, before any processing.

Filename	SHA-256 (first 64 hex chars)	Bytes	Collection Timesta

			mp (WAT)
dataset_small.csv	74dfb1fd05b9d277f1dda62d002cdd3d81ff6ac754d04c8f621633206ab3e56	16,324	2026-10-27 14:34:29
dataset_test.jsonl	7c78cee96cf5f2475bc894713b5b4b5b631b33aaa684665fe2e59663e21e22	4,608	2026-10-27 14:34:29
dataset_train.jsonl	93938b9d9b26b7dd1261d19a8e374c10fe50b5e0733f415572a0fdb8f035c27	21,860	2026-10-27 14:34:29
fine_tuning_steps.txt	863c43f1029fbd6e5e62516e0beb07b544f7a413bc315ccf3c08e782bd60cd	31,869	2026-10-27 14:34:29
result.csv	29b85f57d78515c7c8ad529bee77e91dd1555e4d515d43000120eafb58	41,353	2026-10-27 14:34:29

3. Methodology

3.1 Dataset Profiling (Phase A)

dataset_small.csv was loaded with pandas. Results:

- **Records:** 229
- **Columns:** ['ip', 'class', 'text']
- **Class distribution (source label):** {'A': 177, 'N': 32, 'B': 12, 'C': 4, 'E': 4}
- **Null counts:** 0

- **Duplicate rows:** 77

Strict IPv4 validation (ipaddress.IPv4Address) against source labels produced:

text

strict_valid False True

source

negative 24 8

positive 4 193

This crosstab is forensically significant: **4 source-positive records fail strict IPv4 validation**, and **8 source-negative records pass it**. These disagreements were preserved, not silently relabelled, as required.

JSONL verification: Both files were parsed line-by-line as valid JSON. The split was confirmed: **183 training records** (163 positive, 20 negative) and **46 testing records** (34 positive, 12 negative).

3.2 Candidate Extraction and Validation (Phase B)

A regex candidate extractor (\b(?:\d{1,3}\.){3}\d{1,3}\b) was implemented alongside an independent strict validator using ipaddress.IPv4Address. Original string context was retained for every match.

The candidate_review.csv (229 rows) was exported with the required schema:
source_row, context, candidate, regex_match, ipv4_valid, source_label, analyst_note

Controlled validation tests demonstrated the distinction between three concepts:

Test case	Input	Regex match	Strict IPv4 valid	Interpretation
Genuine IPv4	192.168.1.10	Yes	Yes	Valid candidate
Version-like	v1.2.3.4	Yes	Yes	Syntactically valid but

				version context
Software version	SDKVersion=6.8.4.2	Yes	Yes	Syntactically valid; not IP meaning
Invalid octet	192.168.300.1	Yes	No	Octet exceeds 255
URL-embedded	https://example.test/path/192.168.1.10	Yes	Yes	Embedded address

Dataset category counts from the evidence run:

- Genuine positive IPv4: 191
- Version-like context: 2
- Invalid octet above 255: 25
- URL-embedded candidate: 26
- Valid IPv4 labelled negative: 8
- Positive label but invalid IPv4: 4
- Numeric prefix/suffix contamination: 45

3.3 Offline Classifier and Baseline (Phase C)

A text classifier was built with **TF-IDF character n-grams (1–4)** and **Logistic Regression** (max_iter=2000, class_weight='balanced', random_state=42). Vectorizers were fit only on training records.

A transparent **rule-based baseline** was run on the same holdout: a candidate is labelled positive only if it passes strict IPv4 validation **and** appears as a standalone token in context (not glued to digits/letters/dots).

Leakage risk was recorded: LEAKAGE: 30/46 test prompts are exact duplicates of training prompts. This is a material limitation of the supplied split and is discussed in Section 6.

Package versions and seed were saved to environment.txt.

3.4 Historical Fine-Tuning Interpretation (Phase D)

fine_tuning_steps.txt (734 steps) and result.csv (733 steps) were parsed. Non-null metric counts: classification metrics (accuracy, precision, recall, AUROC, AUPRC, F1) = 4 each; validation_loss and validation_sequence_accuracy = 92 each.

Final non-null recorded values at step 733:

Metric	Final value	Step
classification/accuracy	0.9783	733
classification/precision	0.9714	733
classification/recall	1.0000	733
classification/auroc	0.9975	733
classification/auprc	0.9991	733

Training loss vs. step and accuracy-type metrics vs. step were plotted (historical_training_curves.png). Training loss spikes (e.g., near steps 40, 170, 300) indicate optimisation instability; validation loss tracks training loss without clear divergence, but the sparse classification metric checkpoints (4 values) limit conclusions. **These are historical logs, not a new student-run model**, and cannot be treated as independent holdout performance.

The historical workflow (upload → fine-tuning job → status → prediction → evaluation) used the legacy "ada" model and endpoints with embedded credentials. These API patterns are obsolete and cannot be assumed usable today; no historical upload or prediction scripts were executed.

4. Results (Phase E)

4.1 Holdout Evaluation (n = 46)

Model	TP	FP	FN	TN	Accuracy	Precision	Recall	F1
Majority class	34	12	0	0	0.7391	0.7391	1.0000	0.8500
Rule (regex + ipaddress)	21	2	13	10	0.6739	0.9130	0.6176	0.7368

TF-IDF char + LogReg	34	1	0	11	0.9783	0.9714	1.0000	0.9855

ML AUROC: 0.9902 | **AUPRC:** 0.9965

Per-class classification report (TF-IDF + LogReg):

text

	precision	recall	f1-score	support
negative	1.00	0.92	0.96	12
positive	0.97	1.00	0.99	34
accuracy			0.98	46
macro avg	0.99	0.96	0.97	46
weighted avg	0.98	0.98	0.98	46

Confusion matrices are saved as confusion_matrices.png. The ML model produced **1 false positive and 0 false negatives**; the rule baseline produced **2 false positives and 13 false negatives**.

4.2 Actual-versus-Predicted Table

Every holdout record is saved in out/predictions.csv with columns: prompt, actual, ml_pred_label, ml_score, outcome, rule_pred, majority_pred. Outcome labels (TP/TN/FP/FN) are assigned with "positive" as the positive class.

5. Eight-Record Analyst Review

The following review covers source-labelled negatives, apparent positives, and the identified ML error, drawn from predictions.csv and candidate_review.csv.

Record 1 — Source row 2 (source label: positive; strict IPv4: invalid)
Context: <string>1.000.000.000+</string>. Regex matched 1.000.000.000, which fails strict validation (leading zeros / octet > 255). **Decision:** preserved as a source-positive/strict-invalid disagreement. The candidate is contextually a numeric fragment, not a routable address; analyst note records the disagreement rather than deleting it.

Record 2 — Source row 7 (source label: negative; strict IPv4: invalid)
Context: <string>3608.80.24.1.8</string>. The regex extracted 608.80.24.1, which fails validation. **Decision:** correctly a negative; the dotted sequence is a version/serial-like string. Confirms regex alone would over-collect.

Record 3 — Source row 131 (source label: negative; strict IPv4: valid)
Context: <string>0.0.0.0</string>. Strictly valid but source-labelled negative. **Decision:** retained as a valid-but-negative case; 0.0.0.0 is a wildcard/unspecified address and contextually not an evidence candidate. This is a key false-positive trap for a purely syntactic rule.

Record 4 — Source row 48 (source label: positive; strict IPv4: valid)
Context: http://en-us-support.belkin.com/eu/f/rightnow/js/10.5.0.1.6.137/min/RightNow.js. Candidate 10.5.0.1 is embedded in a longer dotted numeric sequence. **Decision:** source-positive retained; ML correctly classified TP (score 0.728). Rule baseline misclassified as negative (embedded context).

Record 5 — Source row 29 (source label: positive; version-like)
Context: ...SDKVersion=6.8.4.2&initCountry=US.... Candidate 6.8.4.2 is syntactically valid but contextually a software version. **Decision:** source-positive retained; demonstrates why syntactic validity does not establish IP meaning. ML correctly classified as positive; a context-aware rule should flag this.

Record 6 — Source row 206 (source label: negative; ML false positive)
Context: tincan_register_device_256.0.1.26.113. Candidate 256.0.1.26 fails strict validation (octet 256). **ML prediction: positive (FP), score 0.676. Decision:** this is the single ML error. The character n-gram model over-weighted the numeric-dotted pattern despite invalid octets. Analysts must escalate: a false positive here wastes review capacity but carries low forensic risk; the model requires a strict-validator guard.

Record 7 — Prediction index 19, source rows 180/204/205/208/209 (source label: negative; TN)
Context: 257.0.1.38.116. Candidate 257.0.1.38 invalid (octet 257). **ML prediction: negative (TN), score 0.357. Decision:** correctly rejected; combined with Record 6, shows the model is not uniformly reliable on invalid-octet strings.

Record 8 — Prediction index 42, source rows 130/132/227/229 (source label: positive; TP)
Context: <string>255.255.255.255</string>. Candidate 255.255.255.255 is valid. **ML prediction: positive (TP), score 0.731. Decision:** correctly identified; however, 255.255.255.255 is a broadcast address and is not by itself evidence of maliciousness. Retained as a candidate requiring corroboration.

Overall review finding: The ML model's near-perfect holdout score must be read against the 30/46 duplicate leakage. The one FP (invalid-octet string) and the zero FNs suggest the model memorised many exact duplicates rather than generalising. The rule baseline's 13 FNs show that strict validation alone under-collects embedded and version-adjacent candidates.

6. Limitations and Counterexample

6.1 Data Leakage

30 of 46 test prompts are exact duplicates of training prompts. The reported 0.9783 accuracy and 0.9902 AUROC are therefore **optimistic** and cannot be presented as generalisation performance. A strict conclusion is that the model performs well on repeated prompts; a leakage-free split is required before deployment claims.

6.2 Imbalanced Data Counterexample

Training labels are 163 positive : 20 negative; strict-valid counts are 4 positive : 193 negative depending on annotation lens. On the holdout, a **majority-class classifier predicting "positive" for every record** achieves **73.91% accuracy** (34/46) with **1.0000 recall** — yet it would label all 12 negatives as positives and identify no negative evidence at all. High accuracy on an imbalanced dataset is therefore operationally poor: the metric rewards the dominant class. Precision, recall, F1, and the confusion matrix must be reported together.

False-negative consequences for forensic triage: A missed positive (FN) means a candidate IP evidence item is never escalated for corroboration — a potential loss of investigative leads. In this holdout the ML model produced 0 FNs (with leakage caveats); the rule baseline produced 13 FNs, which in an operational setting would suppress 13 of 34 positives. **Escalation policy:** any FN on a source-positive must be flagged for analyst review, and low-confidence positives (score near the decision boundary) should be queued for manual corroboration rather than auto-closed.

6.3 Other Limitations

- Historical fine-tuning logs are sparse (4 classification checkpoints over 733 steps) and cannot be independently reproduced; they are not a new model run.
- The legacy "ada" fine-tuning workflow uses obsolete endpoints and embedded credentials; those scripts were not executed and no tokens appear in this report.
- Source annotations include unusual address strings; strict validity and context-aware evidence classification answer different questions and are both reported.

7. Relative Strengths and Limits

Approach	Strengths	Limits
Regex + strict IPv4 validation	Transparent, reproducible, no training data needed; rejects invalid octets	Over-collects version-like and embedded strings; under-collects (13 FNs); no context reasoning
Offline ML (TF-IDF char + LogReg)	High holdout scores; fast, local, no GPU	Scores inflated by 30/46 duplicate leakage; 1 FP on invalid-octet string; requires validator guard and leakage-free evaluation
Historical fine-tuning	Recorded high metrics (accuracy 0.9783, F1 0.9855)	Not reproducible; legacy API; sparse logs; cannot be treated as independent evidence
Approach	Strengths	Limits
Regex + strict IPv4 validation	Transparent, reproducible, no training data needed; rejects invalid octets	Over-collects version-like and embedded strings; under-collects (13 FNs); no context reasoning

Recommendation: Deploy the offline ML classifier **only as analyst assistance**, with (a) a strict-validator post-filter to suppress invalid-octet false positives, (b) mandatory human review of all ML positives and all source-positive/strict-invalid disagreements, and (c) re-evaluation on a leakage-free split before any operational use. Regex/IPv4 validation should remain the transparent baseline; neither method classifies an address as malicious without corroborating evidence.

8. Conclusion and Next Investigative Actions

1. **Preserve and verify:** Originals are hash-verified and permission-restricted; evidence_manifest.csv records filename, SHA-256, size, WAT/UTC timestamps, transfer source, and examiner.
2. **Candidate extraction:** candidate_review.csv (229 rows) preserves regex matches, strict-validity results, source labels, and analyst notes, including the 4 positive/invalid and 8 negative/valid disagreements.
3. **Reproducible evaluation:** predictions.csv, metrics_summary.csv, confusion matrices, and environment/seed records allow full reproduction of the holdout metrics.
4. **Next actions:**
    - Rebuild the train/test split to eliminate the 30/46 duplicate leakage and re-measure generalisation.
    - Add a strict-IPv4 post-filter to the ML pipeline and re-test on invalid-octet strings (e.g., 256.0.1.26).
    - Corroborate all candidate addresses against device configuration provenance and, where applicable, network logs; do not classify any address as malicious without independent evidence.
    - Formally retire the legacy fine-tuning workflow and document replacement API controls.

9. Executive Briefing (Two-Page Equivalent)

Purpose: Determine whether AI-assisted triage can distinguish candidate IP evidence from misleading strings in a simulated mobile configuration export.

What we did: Preserved and hashed five evidence files; profiled 229 records (183 train / 46 test); built a transparent regex + strict IPv4 baseline and an offline TF-IDF + Logistic Regression classifier; parsed historical fine-tuning logs.

What we found:

- The ML classifier achieved **97.83% accuracy, 100% recall, 97.14% precision** on the holdout (34 TP, 1 FP, 0 FN, 11 TN) — **but 30 of 46 test prompts duplicate training prompts**, so this is an optimistic figure.
- The rule baseline achieved **67.39% accuracy** (21 TP, 2 FP, 13 FN, 10 TN) — strict validation alone under-collects embedded and version-adjacent candidates.
- A majority-class baseline reaches **73.91% accuracy** without identifying any negatives, illustrating why accuracy alone is operationally misleading on imbalanced data.
- Historical fine-tuning logs report accuracy 0.9783 and F1 0.9855 at step 733, but are not reproducible and use obsolete APIs.

Decision: Deploy AI **as analyst assistance only**. No recovered address is classified as malicious without corroborating evidence. Any source-positive that fails strict validation, or that the model flags with low confidence, is escalated to a human analyst.

Next steps: Re-evaluate on a leakage-free split; add strict-validator post-filtering; corroborate candidate addresses against provenance and network logs; retire legacy fine-tuning endpoints.

Appendix A — Deliverables and Reproducibility

Deliverable	Location
Evidence manifest	out/evidence_manifest.csv
Candidate review	out/candidate_review.csv (229 rows, 7 columns)
Predictions (actual vs. predicted)	out/predictions.csv (46 rows)
Metrics summary	out/metrics_summary.csv
Confusion matrices	out/confusion_matrices.png
Historical curves	out/historical_training_curves.png
Historical final metrics	out/historical_final_metrics.csv

Environment	out/environment.txt
Pipeline / scripts	lab01_pipeline.py, lab01_phaseD_historical.py, phase_b_evidence.py, phase_b_controlled_tests.py, investigate.py, octet_check.py, leakage_check.py
Seed	42 (set in pipeline; confirmed by grep -Rn "random_state|random.seed|np.random.seed")
Deliverable	Location

Reproduction command: python lab01_pipeline.py ../originals from the work/ directory.

Appendix B — Controlled validation test evidence is recorded in the terminal transcript (phase_b_controlled_tests.py): genuine IPv4, version-like string, software version, invalid octet, URL-embedded IPv4, and long dotted sequence cases were each tested and are labelled "controlled tests only — not original dataset records."

Appendix C — 12 diverse actual evidence examples were printed by phase_b_evidence.py (12 unique candidates, 12 unique source rows, source records only), covering genuine positives, version-like contexts, invalid octets, URL-embedded candidates, valid-but-negative labels, positive-but-invalid labels, and numeric contamination.